



MANUEL D'UTILISATION DU SIMULATEUR

Apprentissage par renforcement d'une politique de navigation mono-agent avec l'algorithme SAC

SUJET DU STAGE : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

AUTEURE : Sayan Suos

Table des matières

Introduction	2
1 Installation du simulateur	3
1.1 Prérequis	3
1.2 Installation	3
2 Structure du projet	4
2.1 Organisation des fichiers	4
2.2 Architecture du simulateur	5
2.3 Organisation des classes	6
3 Configuration du simulateur	8
3.1 Configuration des expériences	8
3.2 Configuration des environnements	9
3.3 Configuration des agents	12
3.4 Configuration de la fonction de récompense	13
4 Utilisation du simulateur	15
4.1 Modes d'exécution	15
4.2 Entraînement d'une politique	16
4.3 Validation d'une politique	17
4.4 Évaluation d'une politique	18
5 Visualisation des résultats	20
5.1 Démonstration en direct	20
5.2 Génération d'une figure comparative des scénarios	20
5.3 Génération d'une animation	21
5.4 Résultats générés	22
Glossaire	24
FAQ	29
Comment entraîner une nouvelle politique ?	29
Quelle est la différence entre la validation et l'évaluation ?	29
Quand utiliser <code>run_demo</code> ou <code>run_animation</code> ?	29
À quoi sert le paramètre <code>trained_agent_id</code> ?	29
Où sont enregistrés les résultats ?	29
Comment reprendre un entraînement ?	29
Pourquoi certaines configurations semblent-elles ignorer le nombre d'obstacles ?	30
Comment ajouter un nouveau scénario ?	30

Introduction

Le transport de bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire soulève des problématiques complexes de navigation autonome, de coordination entre agents et d'évitement d'obstacles dans des environnements dynamiques. Afin d'étudier ces problématiques, un simulateur a été développé pour modéliser différents scénarios de navigation et expérimenter des approches d'apprentissage par renforcement.

Dans une première étape, le simulateur est utilisé dans un contexte plus général de navigation de robots mobiles autonomes (AMR). Il permet de créer des environnements configurables composés d'agents, d'obstacles statiques et d'obstacles dynamiques, puis d'entraîner et d'évaluer des politiques de navigation. La version actuelle du simulateur repose sur l'algorithme Soft Actor-Critic (SAC) et se concentre sur l'apprentissage d'un agent unique dans des environnements de difficulté croissante.

Ce manuel a pour objectif de présenter l'utilisation du simulateur et d'accompagner l'utilisateur dans sa prise en main. Il décrit successivement les étapes d'installation, l'organisation générale du projet, les fichiers de configuration, les principaux modes d'exécution ainsi que les outils permettant de visualiser et d'analyser les résultats générés.

Ce document se concentre volontairement sur les aspects pratiques liés à l'utilisation du simulateur. Les choix de modélisation, l'algorithme Soft Actor-Critic ainsi que le protocole expérimental sont détaillés dans le document de modélisation associé.

1 Installation du simulateur

Cette section décrit les différentes étapes nécessaires pour installer le simulateur sur une machine locale. Les procédures présentées ont été testées sous macOS et Windows.

1.1 Prérequis

Avant d'installer le simulateur, les logiciels suivants doivent être installés sur la machine :

- Python (version 3.11 ou supérieure) ;
- Git, utilisé pour cloner le dépôt du projet ;
- un environnement virtuel Python.

1.2 Installation

Le projet est distribué sous la forme d'un dépôt Git. La première étape consiste donc à se placer dans le dossier souhaité, puis à cloner le dépôt sur la machine locale :

```
1 git clone https://github.com/sayansuos/baggage-handling-rl.git
2 cd baggage-handling-rl
```

Il est ensuite recommandé de créer un environnement virtuel afin d'isoler les dépendances du projet. Celui-ci peut ensuite être activé selon le système d'exploitation utilisé :

- Sous macOS ou Linux :

```
1 python -m venv .venv
2 source ./venv/bin/activate
```

- Sous Windows :

```
1 python -m venv .venv
2 .\venv\Scripts\activate
```

Une fois l'environnement virtuel activé, les dépendances du projet peuvent être installées à partir du fichier `requirements.txt` :

```
1 python -m pip install -r requirements.txt
```

Une fois cette étape terminée, le simulateur est prêt à être utilisé.

2 Structure du projet

Cette section présente l'organisation générale du projet ainsi que l'architecture du simulateur. Elle décrit tout d'abord la structure des principaux fichiers et dossiers du dépôt, puis le fonctionnement global du simulateur au travers de son architecture fonctionnelle. Enfin, elle introduit les principales classes constituant le simulateur ainsi que leurs relations afin de faciliter la compréhension de son implémentation.

2.1 Organisation des fichiers

Le projet est organisé en plusieurs dossiers, chacun regroupant un ensemble de fonctionnalités spécifiques. Le rôle des principaux fichiers et dossiers est résumé dans le Tableau 1.

TABLE 1 – Description des principaux dossiers et fichiers du projet.

<i>Élément</i>	<i>Description</i>
<code>configs/</code>	Fichiers de configuration du simulateur, regroupant les paramètres des environnements, des agents, des fonctions de récompense et des expériences d'entraînement.
<code>docs/</code>	Documentation du projet, comprenant le manuel d'utilisation, le document de modélisation ainsi que les comptes rendus du projet.
<code>figures/</code>	Figures, graphiques et animations générés lors de l'entraînement et de l'évaluation du modèle.
<code>logs/</code>	Métriques générés lors de l'entraînement et de l'évaluation du modèle.
<code>rl/</code>	Implémentation des algorithmes d'apprentissage par renforcement ainsi que des réseaux de neurones associés.
<code>simulator/</code>	Cœur du simulateur. Regroupe les principaux composants de la simulation (environnement, agents, obstacles), les outils nécessaires à l'apprentissage par renforcement (observations, fonction de récompense), la gestion des interactions entre les entités ainsi que le calcul des trajectoires des obstacles dynamiques par A*.
<code>utils/</code>	Fonctions utilitaires utilisées par l'ensemble du projet (visualisation, journalisation, chargement des configurations, etc.).
<code>main.py</code>	Point d'entrée du projet, permettant de sélectionner le mode d'exécution du simulateur.
<code>runner.py</code>	Fonctions responsables du lancement des entraînements, des validations, des évaluations et de la génération des animations.

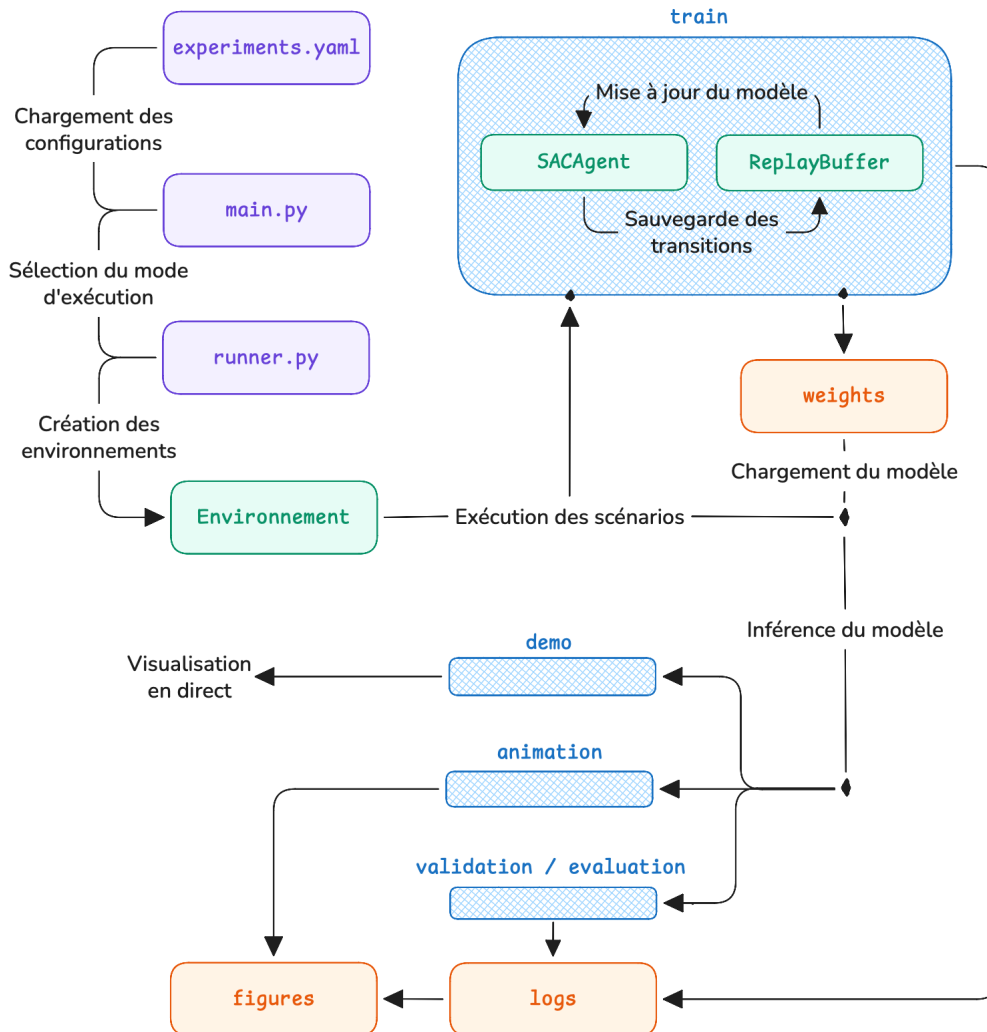
Remarque

Le dossier `simulator/` contient l'ensemble des composants nécessaires au fonctionnement du simulateur. Dans la plupart des cas, l'utilisateur n'a pas besoin de modifier ce dossier : la personnalisation du comportement du simulateur s'effectue principalement à travers les fichiers de configuration présentés dans la section 3.

2.2 Architecture du simulateur

La Figure 1 présente l'architecture fonctionnelle du simulateur ainsi que les principales étapes d'exécution d'une expérience.

FIGURE 1 – Architecture fonctionnelle du simulateur



Quel que soit le mode d'exécution sélectionné (`train`, `validation`, `evaluation`, `animation` ou `demo`), le programme débute dans le fichier `main.py`. Celui-ci charge les fichiers de configuration afin d'initialiser l'environnement de simulation, les agents ainsi que les scénarios définis par l'utilisateur. Il sélectionne ensuite le mode d'exécution souhaité puis appelle les fonctions correspondantes du fichier `runner.py`.

En mode `train`, un agent est entraîné à l'aide de l'algorithme Soft Actor-Critic (SAC). À chaque interaction avec l'environnement, les observations de l'environnement sont transmises à l'algorithme SAC, qui sélectionne une action ensuite appliquée à l'environnement. Cela produit un nouvel état et une récompense. Les transitions sont stockées dans le `Replay Buffer` et utilisées pour mettre à jour les réseaux de neurones. À la fin de l'entraînement, les poids du modèle ainsi que les différentes métriques sont enregistrés.

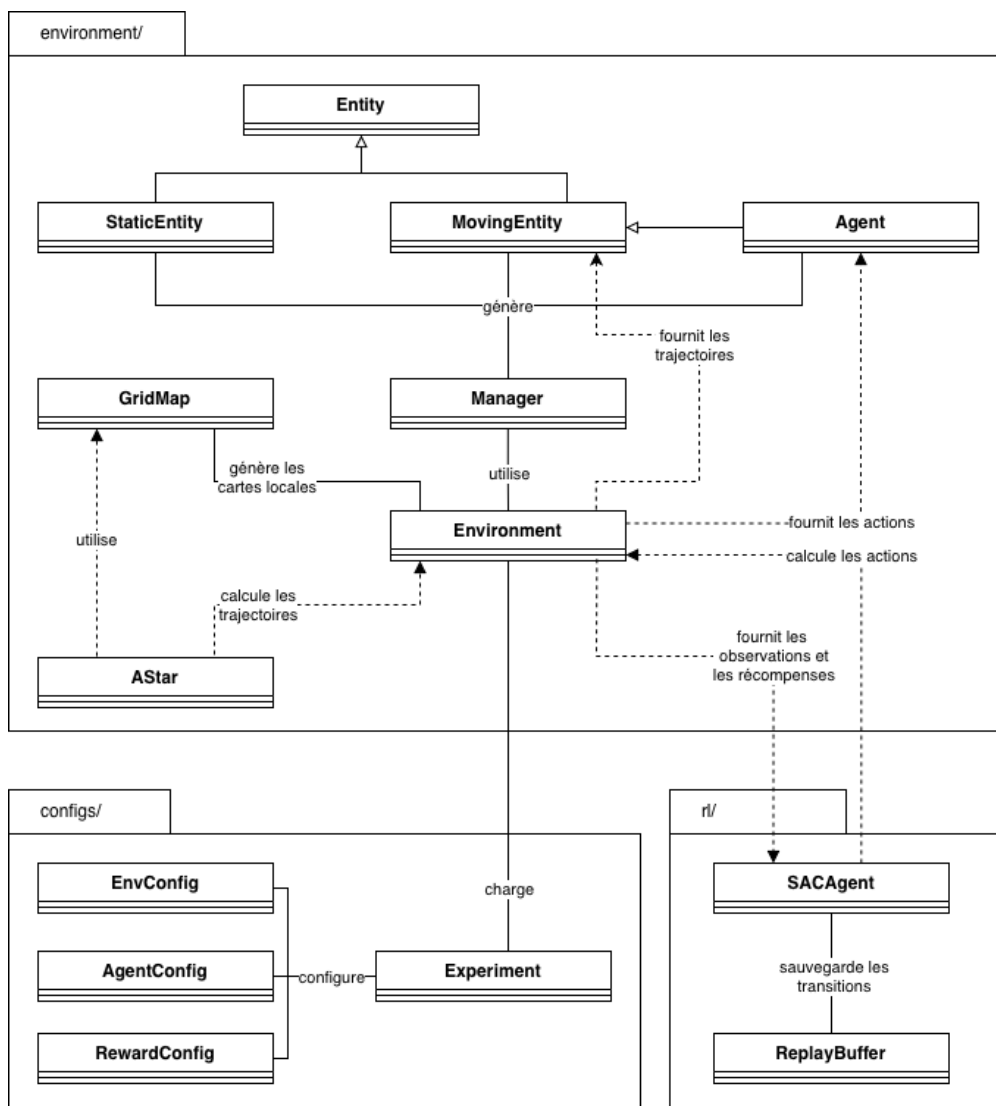
Les modes `validation`, `evaluation`, `animation` et `demo` utilisent quant à eux un modèle préa-

lablement entraîné. Les poids sont chargés afin de calculer les actions de l'agent. Selon le mode sélectionné, le simulateur produit ensuite des métriques, des figures, une animation ou une visualisation en direct.

2.3 Organisation des classes

Le simulateur repose sur une architecture orientée objet dans laquelle les différents éléments présents dans un environnement sont représentés par des classes. La Figure 2 présente un diagramme de classes simplifié des principaux composants du simulateur.

FIGURE 2 – Diagramme de classes simplifié du simulateur.



Remarque

Le diagramme de classes présenté est volontairement simplifié. Les attributs et les méthodes des différentes classes n'y sont pas représentés afin de privilégier une vue d'ensemble de l'architecture du simulateur et des principales relations entre ses composants.

La classe `Entity` constitue la classe de base des objets présents dans l'environnement. Elle définit les propriétés communes aux différentes entités, notamment leur position, leurs dimensions ainsi que les mécanismes de détection des collisions.

Deux classes héritent de cette celle-ci : la classe `StaticEntity` qui représente les obstacles statiques de l'environnement, ainsi que la classe `MovingEntity` qui représente les entités capables de se déplacer, telles que des obstacles dynamiques. La classe `Agent` hérite de cette dernière et complète son comportement avec les propriétés nécessaires à la navigation et à l'apprentissage par renforcement.

La classe `Environment` regroupe les agents, les obstacles statiques et les obstacles dynamiques qui composent un scénario. Elle est responsable de leur initialisation (via la classe `Manager`), de l'application des actions et trajectoires, de la mise à jour de la simulation, du calcul des observations et des récompenses, ainsi que de la détection de la fin d'un épisode.

La classe `GridMap` fournit une représentation discrète de l'environnement. Elle est utilisée à la fois pour construire les cartes locales observées par les agents et pour calculer les trajectoires des obstacles dynamiques à l'aide de la classe `AStar`.

Enfin, l'apprentissage par renforcement est assuré par la classe `SACAgent`, qui s'appuie sur un `ReplayBuffer` ainsi que sur les réseaux de neurones `FeaturesExtractor`, `ActorNetwork` et `CriticNetwork`. Les paramètres des environnements, des agents et de la fonction de récompense sont quant à eux définis respectivement par les classes `EnvironmentConfig`, `AgentConfig` et `RewardConfig`.

3 Configuration du simulateur

La configuration du simulateur repose sur un ensemble de fichiers YAML regroupés dans le dossier `configs/`. Ces fichiers permettent de définir l'ensemble des paramètres nécessaires à l'exécution d'une expérience, notamment les caractéristiques des environnements, des agents, de la fonction de récompense ainsi que les expériences d'entraînement ou d'évaluation. Cette séparation entre la configuration et le code permet de modifier facilement le comportement du simulateur sans intervenir directement dans son implémentation.

TABLE 2 – Description des fichiers de configuration.

Élément	Description
<code>config.py</code>	Définit les différentes classes de configuration (dataclasses) utilisées pour représenter les paramètres des environnements, des agents, des fonctions de récompense et des expériences après le chargement des fichiers YAML.
<code>experiments.yaml</code>	Définit les expériences à exécuter.
<code>environments.yaml</code>	Définit les environnements.
<code>agent.yaml</code>	Définit les paramètres des agents.
<code>reward.yaml</code>	Définit les paramètres de la fonction de récompense.

Remarque

Les fichiers de configuration sont automatiquement chargés par le module `config_loader` (dans le dossier `utils/`), qui lit les fichiers YAML et instancie les différentes classes de configuration (`EnvironmentConfig`, `AgentConfig`, `RewardConfig` et `Experiment`). L'utilisateur n'a donc qu'à modifier les fichiers de configuration sans intervenir dans le code source.

Les sections suivantes détaillent le contenu de chaque fichier de configuration.

3.1 Configuration des expériences

Les expériences à exécuter sont définies dans le fichier `experiments.yaml`. Une expérience correspond à un scénario d'entraînement ou d'évaluation et associe un environnement, une configuration d'agent ainsi qu'une fonction de récompense. Ce fichier permet également de définir le nombre d'étapes d'entraînement.

Chaque expérience d'entraînement est définie par une entrée de la liste `train_experiments`.

```
1 train_experiments:
2   - name: random_1
3     env_config: random_1
4     agent_config: agent_1
5     reward_config: reward_1
6     n_steps: 50_000
```

De même, chaque expérience d'évaluation est définie par une entrée de la liste `eval_experiments`.

```
1 eval_experiments:
2   - name: random_1
3     env_config: random_1
4     agent_config: agent_1
5     reward_config: reward_1
6     n_steps: null
```

TABLE 3 – Description des paramètres de Experiment.

Paramètres	Description
name	Nom de l'expérience. Ce nom est également utilisé pour identifier les fichiers de résultats et les poids sauvegardés.
env_config	Nom de la configuration d'environnement (définie dans <code>environment.yaml</code>).
agent_config	Nom de la configuration de l'agent (définie dans <code>agent.yaml</code>).
reward_config	Nom de la fonction de récompense (définie dans <code>reward.yaml</code>).
n_steps	Nombre d'étapes d'entraînement.

Les expériences sont exécutées dans l'ordre où elles apparaissent, ce qui permet notamment de mettre en place un curriculum d'apprentissage. Chaque expérience peut ainsi utiliser une configuration d'environnement différente afin d'augmenter progressivement la difficulté des scénarios, tandis que les poids du modèle appris lors d'une expérience sont réutilisés pour initialiser la suivante.

Remarque

Les expériences d'entraînement sont exécutées pendant un nombre d'étapes défini par le paramètre `n_steps`. Ce nombre doit être choisi en fonction de la difficulté du scénario ainsi que de la diversité des situations rencontrées durant l'apprentissage.

Les expériences d'évaluation n'utilisent pas ce paramètre. Elles sont exécutées sur un nombre fixe d'épisodes afin d'évaluer la politique préalablement entraînée. Ce nombre d'épisodes est défini dans la fonction `run_evaluation` présentée dans la section 4.4.

3.2 Configuration des environnements

Les environnements sont définis dans le fichier `environments.yaml`. Chaque environnement décrit un scénario de simulation en précisant les dimensions de la carte, les modes de génération des entités ainsi que les paramètres contrôlant la création des obstacles et le déroulement d'un épisode.

Chaque environnement est identifié par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l'aide du paramètre `env_config`.

```

1  random_1:
2    width: 64
3    height: 32
4
5    agent_mode: random
6    env_mode: random
7
8    nb_agents: 1
9    nb_static_obstacles: 4
10   nb_moving_obstacles: 2
11   nb_targets: 2
12
13   margin: 3
14   max_attempts: 100
15   max_steps: 300
16
17   thickness: 1
18   radius_min: 0.5
19   radius_max: 1.0

```

TABLE 4 – Description des paramètres de `EnvironmentConfig`.

<i>Paramètre</i>	<i>Description</i>
<code>width</code>	Largeur de l’environnement.
<code>height</code>	Hauteur de l’environnement.
<code>agent_mode</code>	Définit le mode de génération des agents et de leurs objectifs.
<code>env_mode</code>	Définit le type d’environnement et la disposition des obstacles statiques.
<code>nb_agents</code>	Nombre d’agents présents dans le scénario.
<code>nb_static_obstacles</code>	Nombre d’obstacles statiques générés, lorsque le mode d’environnement le permet.
<code>nb_moving_obstacles</code>	Nombre d’obstacles dynamiques générés.
<code>nb_targets</code>	Nombre de positions cibles successives attribuées à chaque agent ou obstacle dynamique.
<code>margin</code>	Distance minimale imposée entre les entités lors de leur génération.
<code>max_attempts</code>	Nombre maximal de tentatives pour générer une position valide avant l’abandon.
<code>max_steps</code>	Nombre maximal d’étapes autorisées pour un épisode.
<code>thickness</code>	Épaisseur des murs délimitant l’environnement.
<code>radius_min</code>	Rayon minimal des obstacles dynamiques générés aléatoirement.
<code>radius_max</code>	Rayon maximal des obstacles dynamiques générés aléatoirement.

Les paramètres `env_mode` et `agent_mode` déterminent la stratégie de génération des scénarios. Selon les valeurs choisies, les positions des agents, des obstacles statiques, des obstacles dynamiques et des cibles sont générées suivant des règles différentes, permettant de construire des environnements adaptés à différents niveaux de difficulté. Le code source de ces configurations se trouvent dans la classe `Manager`.

TABLE 5 – Description des modes de génération.

<i>Paramètre</i>	<i>Valeur</i>	<i>Description</i>
<code>env_mode</code>	<code>random</code>	Génère aléatoirement les positions et les dimensions des obstacles statiques dans l’environnement.
	<code>warehouse</code>	Génère une disposition prédéfinie d’obstacles statiques représentant des allées d’entrepôt.
	<code>crossing</code>	Génère les obstacles statiques autour du centre de l’environnement selon une disposition circulaire.
	<code>fixed_n</code>	Génère les obstacles statiques sur <code>n</code> colonnes fixes, avec des dimensions déterministes.
	<code>fixed_random_n</code>	Génère les obstacles statiques sur <code>n</code> colonnes fixes, avec des dimensions aléatoires.
<code>agent_mode</code>	<code>random</code>	Génère aléatoirement les positions initiales et les cibles des agents et des obstacles dynamiques, en respectant les contraintes de distance minimale.
	<code>fixed_n</code>	Génère les positions initiales et les cibles des agents de part et d’autre des <code>n</code> colonnes d’obstacles statiques. Les obstacles dynamiques sont placés dans la partie supérieure ou inférieure de l’environnement, avec des cibles situées dans la zone opposée.
	<code>crossing</code>	Génère les positions initiales des agents et des obstacles dynamiques sur un cercle, puis leur attribue une cible diamétralement opposée afin de créer un scénario de croisement.

Selon le mode de génération sélectionné, certains paramètres de la configuration peuvent être ignorés. Par exemple, le mode `warehouse` repose sur une disposition prédéfinie des obstacles statiques et ne permet donc pas de contrôler directement leur nombre ou leurs dimensions. De même, dans les modes `fixed_n`, `fixed_random_n` et `crossing`, les dimensions des obstacles sont déterminées automatiquement à partir des dimensions de l’environnement et du nombre d’obstacles demandé. Pour le mode `random`, les dimensions maximales des obstacles sont également adaptées.

Remarque

Les différents modes de génération ont été conçus pour produire des scénarios favorables à l'apprentissage par renforcement. Les positions des agents, des obstacles et des cibles ne sont donc pas uniquement générées de manière aléatoire, mais suivent des règles permettant de reproduire les situations recherchées (croisement, franchissement d'obstacles, etc.). Cette méthode favorise des interactions pertinentes entre les agents et leur environnement, tout en garantissant une diversité suffisante des scénarios rencontrés durant l'apprentissage.

3.3 Configuration des agents

Les caractéristiques des agents sont définies dans le fichier `agents.yaml`. Cette configuration regroupe les paramètres physiques du robot ainsi que les paramètres définissant son espace des observations et son espace des actions.

Chaque configuration d'agent est identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l'aide du paramètre `agent_config`.

```
1 agent_1:
2   n_maps: 3
3
4   v_min: 0.0
5   v_max: 5.0
6
7   omega_min: -1.047
8   omega_max: 1.047
9
10  radius: 0.5
11  length_view: 11
12
13  collision_threshold: 0.5
14  reach_threshold: 0.5
```

Les paramètres `v_min`, `v_max`, `omega_min` et `omega_max` définissent directement les bornes de l'espace des actions de l'agent. Les commandes calculées par l'algorithme d'apprentissage sont automatiquement ramenées dans ces intervalles avant d'être appliquées au simulateur.

Remarque

Les paramètres `n_maps` et `length_view` contrôlent la quantité d'informations spatiales et temporelles disponible pour l'agent. Une carte locale plus grande ou un nombre plus important de cartes successives permettent de représenter plus précisément l'environnement et l'évolution des obstacles, mais augmentent également la dimension des observations traitées par l'algorithme d'apprentissage.

TABLE 6 – Description des paramètres de AgentConfig.

<i>Paramètre</i>	<i>Description</i>
n_maps	Nombre de cartes locales successives conservées dans l’observation afin d’intégrer une information temporelle.
v_min	Vitesse linéaire minimale autorisée pour l’agent.
v_max	Vitesse linéaire maximale autorisée pour l’agent.
omega_min	Vitesse angulaire minimale autorisée pour l’agent.
omega_max	Vitesse angulaire maximale autorisée pour l’agent.
radius	Rayon de l’agent utilisé pour la détection des collisions.
length_view	Taille de la carte locale observée par l’agent.
collision_threshold	Distance à partir de laquelle l’agent est considéré en collision avec un obstacle ou un autre agent.
reach_threshold	Distance maximale entre l’agent et sa cible pour considérer que celle-ci est atteinte.

3.4 Configuration de la fonction de récompense

Les paramètres de la fonction de récompense sont définis dans le fichier `rewards.yaml`. Cette configuration permet d’ajuster l’importance des différentes composantes de la récompense utilisées durant l’apprentissage, telles que la progression vers la cible, l’évitement des obstacles ou la limitation des rotations inutiles.

Chaque configuration de récompense est identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l’aide du paramètre `reward_config`.

```

1  reward_1:
2    beta1: 5.0
3    beta2: 0.1
4    beta3: 0.5
5
6    linear_bonus_factor: 2.0
7    progress_reduction_factor: 0.2
8    angular_malus_factor: -2.0
9    safety_malus_factor: -1.0
10
11   goal_bonus: 400.0
12   step_malus: -1
13   collision_malus: -100.0
14
15   safety_threshold: 2.0

```

TABLE 7 – Description des paramètres de RewardConfig.

<i>Paramètre</i>	<i>Description</i>
beta1	Pondération du terme de récompense lié à la progression vers la cible.
beta2	Pondération du terme de récompense lié à la rotation de l'agent.
beta3	Pondération du terme de récompense lié au respect des distances de sécurité.
linear_bonus_factor	Coefficient appliqué au bonus proportionnel à la vitesse linéaire de l'agent.
progress_reduction_factor	Facteur réduisant la récompense de progression lorsque l'agent se trouve à proximité d'un obstacle.
angular_malus_factor	Pénalité appliquée en fonction de la vitesse angulaire de l'agent.
safety_malus_factor	Pénalité appliquée lorsque l'agent évolue à une distance inférieure au seuil de sécurité.
goal_bonus	Récompense attribuée lorsque l'agent atteint sa cible.
step_malus	Pénalité appliquée à chaque étape afin d'encourager des trajectoires plus courtes.
collision_malus	Pénalité appliquée lorsqu'une collision est détectée.
safety_threshold	Distance de sécurité à partir de laquelle la pénalité de sécurité est appliquée.

Les paramètres de cette configuration permettent d'ajuster l'importance relative des différentes composantes de la récompense. En modifiant les coefficients de pondération ou les bonus et pénalités associés à certains comportements, il est possible d'orienter l'apprentissage vers des politiques privilégiant, par exemple, une navigation plus rapide, plus prudente ou plus fluide.

La formulation mathématique complète de la fonction de récompense ainsi que le rôle de chacune de ses composantes sont présentés dans le document de modélisation.

Remarque

La fonction de récompense influence directement le comportement appris par l'agent. Une modification importante de ses paramètres modifie l'objectif d'optimisation de l'algorithme d'apprentissage et nécessite un nouvel entraînement complet du modèle.

4 Utilisation du simulateur

Cette section décrit les principaux modes d'exécution du simulateur ainsi que la procédure permettant d'entraîner, de valider et d'évaluer une politique d'apprentissage par renforcement. Elle présente également le déroulement général d'une expérience, depuis son lancement jusqu'à la production des résultats.

4.1 Modes d'exécution

Le simulateur propose plusieurs modes d'exécution permettant d'entraîner un agent, d'évaluer une politique existante ou encore de générer des visualisations des résultats. Le mode d'exécution est sélectionné dans le fichier `main.py` avant le lancement du programme, en modifiant la valeur de `MODE`.

TABLE 8 – Description des modes d'exécution.

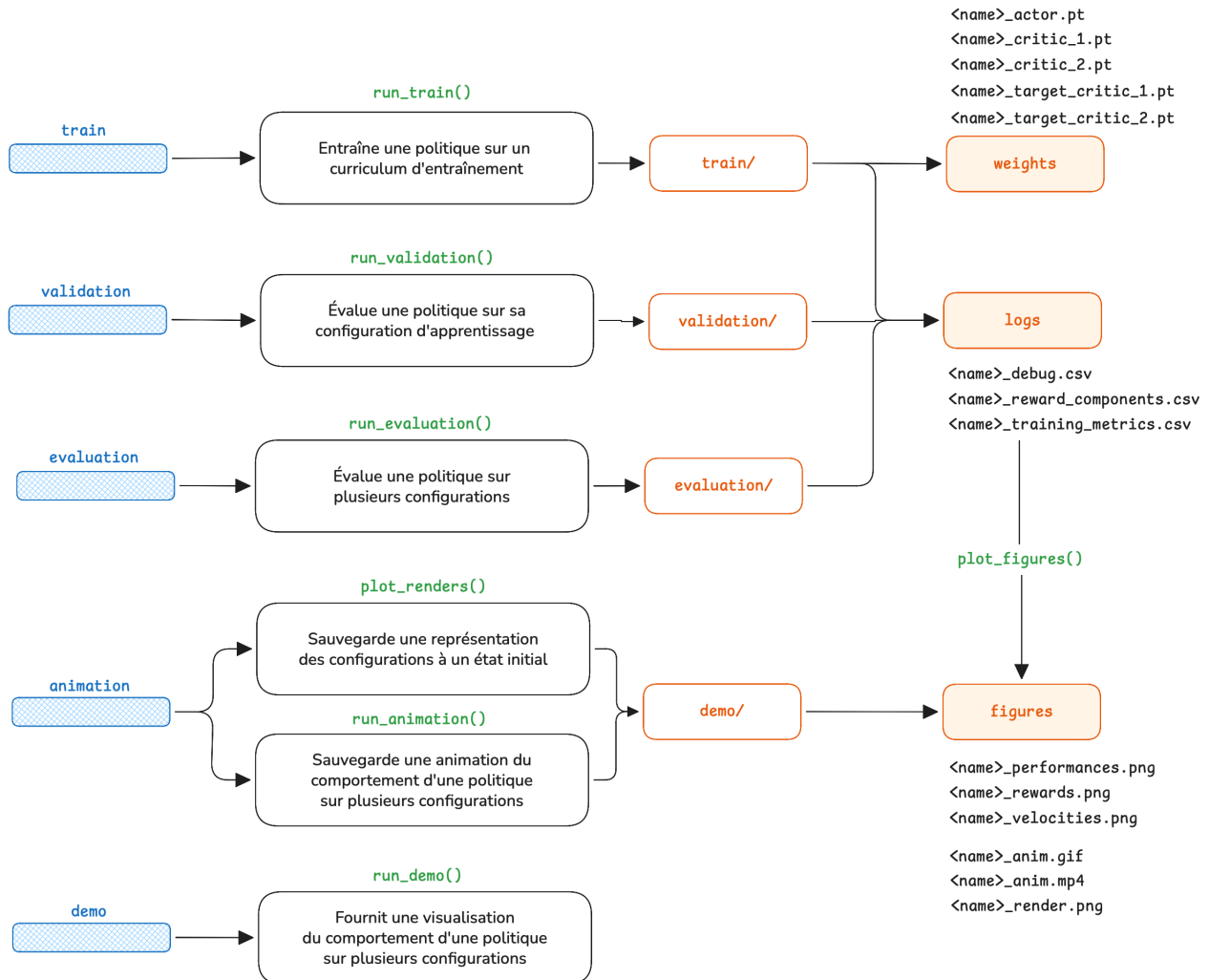
MODE = ...	Description
<code>train</code>	Entraîne successivement un agent SAC sur l'ensemble des expériences du curriculum d'entraînement. À la fin de chaque expérience, les poids du modèle sont sauvegardés avant d'être utilisés comme point de départ de l'expérience suivante. Les métriques et les figures associées sont ensuite sauvegardées afin d'analyser les performances de l'entraînement.
<code>validation</code>	Charge chaque politique entraînée et l'évalue uniquement sur le scénario correspondant à son entraînement. Les métriques et les figures associées sont ensuite sauvegardées afin d'analyser les performances de la politique sur son scénario d'apprentissage.
<code>evaluation</code>	Charge une politique entraînée et l'évalue sur l'ensemble des scénarios du curriculum d'évaluation. Ce mode permet d'étudier la capacité de généralisation des politiques apprises. Les métriques et les figures correspondantes sont sauvegardées.
<code>demo</code>	Charge une politique entraînée et affiche une visualisation en temps réel d'une exécution pour chaque expérience des curriculums d'entraînement et d'évaluation.
<code>animation</code>	Charge une politique entraînée et génère, pour chaque expérience des curriculums d'entraînement et d'évaluation, une animation de l'exécution (GIF et MP4) ainsi qu'une représentation de la configuration de l'environnement.

Après avoir configuré les expériences et sélectionné le mode d'exécution souhaité dans le fichier `main.py`, le simulateur est lancé depuis la racine du projet.

```
1 python main.py
```

La Figure 3 résume le déroulement classique d’une expérience, depuis la configuration des scénarios jusqu’à l’exploitation des résultats.

FIGURE 3 – Workflow d’exécution et organisation des résultats générés par le simulateur.



La Figure 3 résume les principaux traitements réalisés par chaque mode d’exécution. Après la sélection d’un mode dans `main.py`, la fonction correspondante de `runner.py` est exécutée. Selon le mode choisi, le simulateur entraîne une politique, l’évalue sur un ou plusieurs scénarios, ou génère des visualisations. Les résultats obtenus sont ensuite automatiquement enregistrés dans les dossiers dédiés : les poids des réseaux dans `rl/SAC/weights/`, les métriques dans `logs/` et les figures ou animations dans `figures/`.

Les modes dédiés à la visualisation des résultats (`demo` et `animation`) sont décrits dans la section 5.

4.2 Entraînement d’une politique

La fonction `run_train` entraîne successivement une politique sur les expériences du curriculum d’entraînement. À la fin de chaque expérience, les poids du modèle, les métriques d’apprentissage et les figures associées sont sauvegardés.

```

1  def run_train(
2      experiment: list[Experiment],
3      previous_exp: Experiment | None = None,
4      trained_agent_id: str = "agent_1",
5  ) -> None:

```

TABLE 9 – Paramètres de la fonction `run_train`.

Paramètre	Description
<code>experiments</code>	Liste des expériences constituant le curriculum d'entraînement.
<code>previous_exp</code>	Expérience dont les poids doivent être chargés avant le début de l'entraînement. Lorsque la valeur est <code>None</code> , la première politique est initialisée sans poids provenant d'une expérience précédente.
<code>trained_agent_id</code>	Identifiant de l'agent dont la politique doit être entraînée.

Les expériences sont exécutées dans l'ordre de la liste. Les poids obtenus à la fin d'une expérience sont alors utilisés pour initialiser la suivante, ce qui permet de mettre en œuvre un curriculum d'apprentissage.

Remarque

Le paramètre `trained_agent_id` doit correspondre à l'identifiant de l'agent utilisé lors de l'entraînement de la politique. Si plusieurs agents ont été entraînés indépendamment, cet identifiant permet de charger les poids associés à l'agent souhaité.

4.3 Validation d'une politique

La fonction `run_validation` évalue chaque politique sur le scénario correspondant à celui sur lequel elle a été entraînée. Elle sauvegarde les métriques, les figures et, lorsque cela est demandé, plusieurs épisodes sous forme d'animation.

```

1  def run_validation(
2      experiments: list[Experiment]
3      n_episodes: int = 100,
4      n_render: int = 5,
5      trained_agent_id: str = "agent_1",
6  ) -> None:

```

TABLE 10 – Paramètres de la fonction `run_validation`.

<i>Paramètre</i>	<i>Description</i>
<code>experiments</code>	Liste des expériences d’entraînement à valider. Chaque politique est évaluée sur le scénario correspondant à son apprentissage.
<code>n_episodes</code>	Nombre d’épisodes exécutés pour estimer les performances de chaque politique.
<code>n_render</code>	Nombre d’épisodes dont les images sont conservées afin de produire une animation. Une valeur égale à 0 désactive la génération des animations.
<code>trained_agent_id</code>	Identifiant de l’agent contrôlé par la politique évaluée.

Cette étape est essentielle pour suivre l’évolution de l’apprentissage au cours du curriculum. Elle permet de vérifier que chaque nouvelle politique maîtrise effectivement le scénario sur lequel elle a été entraînée, d’évaluer les gains de performance apportés par chaque expérience et de s’assurer que les comportements recherchés sont correctement appris avant de passer à des scénarios plus complexes.

4.4 Évaluation d’une politique

La fonction `run_evaluation` évalue une politique donnée sur l’ensemble des scénarios du curriculum d’évaluation. Contrairement à la validation, la même politique est donc appliquée à plusieurs environnements afin d’étudier sa capacité de généralisation.

```

1 run_evaluation(
2     experiments: list[Experiment],
3     policy: Experiment,
4     n_episodes: int = 100,
5     n_render: int = 5,
6     trained_agent_id: str = "agent_1",
7 ) -> None:

```

TABLE 11 – Paramètres de la fonction `run_evaluation`.

<i>Paramètre</i>	<i>Description</i>
<code>experiments</code>	Liste des scénarios sur lesquels la politique doit être évaluée. Il s’agit généralement de la liste <code>eval_experiments</code> .
<code>policy</code>	Expérience d’entraînement identifiant les poids de la politique à charger.
<code>n_episodes</code>	Nombre d’épisodes exécutés pour chaque scénario d’évaluation.

<i>Paramètre</i>	<i>Description</i>
<code>n_render</code>	Nombre d'épisodes enregistrés sous forme d'animation pour chaque scénario. Une valeur égale à 0 désactive cette génération.
<code>trained_agent_id</code>	Identifiant de l'agent contrôlé par la politique évaluée.

Cette étape vise à évaluer la robustesse et la capacité de généralisation de la politique entraînée. En testant une même politique sur plusieurs scénarios de difficulté variable, il devient possible de vérifier qu'elle conserve les comportements appris dans les environnements les plus simples tout en étant capable de résoudre des situations plus complexes.

5 Visualisation des résultats

Cette section présente les différents outils permettant de visualiser les résultats produits par le simulateur. Elle décrit les fonctions permettant d’observer l’exécution d’une politique en direct, de générer des animations, de représenter graphiquement les scénarios ainsi que les principaux fichiers générés au cours des différentes phases d’exécution.

5.1 Démonstration en direct

La fonction `run_demo` exécute un épisode pour chacune des expériences fournies. L’évolution des agents et des obstacles dynamiques est affichée en temps réel.

```
1 def run_demo(  
2     experiments: list[Experiment],  
3     policy: Experiment,  
4     trained_agent_id: str = "agent_1",  
5 ) -> None:
```

TABLE 12 – Paramètres de la fonction `run_demo`.

<i>Paramètre</i>	<i>Description</i>
<code>experiments</code>	Liste des scénarios à exécuter. Un épisode est lancé pour chaque expérience de la liste.
<code>policy</code>	Expérience d’entraînement identifiant les poids de la politique à charger.
<code>trained_agent_id</code>	Identifiant de l’agent contrôlé par la politique chargée.

La fonction `run_demo` est principalement utilisée pour observer rapidement le comportement d’une politique entraînée sur plusieurs scénarios. Elle permet de visualiser les déplacements des agents, leurs interactions avec les obstacles ainsi que leur progression vers les objectifs, ce qui facilite l’analyse qualitative de la politique apprise et l’identification de comportements inattendus.

5.2 Génération d’une figure comparative des scénarios

La fonction `plot_renders` génère une représentation de l’état initial de chaque expérience fournie.

```
1 def plot_renders(  
2     experiments: list[Experiment],  
3     path: str,  
4     file_name: str = "",  
5     max_ncols: int = 2,  
6 ) -> None:
```

TABLE 13 – Paramètres de la fonction `plot_renders`.

<i>Paramètre</i>	<i>Description</i>
<code>experiments</code>	Liste des expériences dont les environnements doivent être représentés. Une sous-figure est générée pour chaque expérience.
<code>path</code>	Dossier dans lequel la figure comparative doit être enregistrée. pas.
<code>file_name</code>	Préfixe utilisé pour nommer le fichier généré. La figure est enregistrée sous la forme <code><file_name>_renders.png</code> .
<code>max_ncols</code>	Nombre maximal de colonnes utilisé pour organiser les sous-figures.

Cette fonction est principalement utilisée pour obtenir une vue d'ensemble des scénarios d'entraînement et d'évaluation. Elle permet de les illustrer dans des documents tels que le rapport de modélisation. Elle permet également de vérifier visuellement que les scénarios générés correspondent aux configurations définies.

5.3 Génération d'une animation

La fonction `run_animation` exécute un épisode pour chaque scénario, enregistre les images successives du rendu puis les rassemble dans une animation.

```

1  def run_animation(
2      experiments: list[Experiment],
3      policy: Experiment,
4      path: str,
5      file_name: str,
6      fps: int = 20,
7      trained_agent_id: str = "agent_1",
8  ) -> None:

```

TABLE 14 – Paramètres de la fonction `run_animation`.

<i>Paramètre</i>	<i>Description</i>
<code>experiments</code>	Liste des scénarios à exécuter et à inclure dans l'animation.
<code>policy</code>	Expérience d'entraînement identifiant les poids de la politique à charger.
<code>path</code>	Dossier dans lequel les fichiers d'animation doivent être enregistrés.
<code>file_name</code>	Nom utilisé pour les fichiers générés.
<code>fps</code>	Nombre d'images affichées par seconde dans l'animation produite.
<code>trained_agent_id</code>	Identifiant de l'agent contrôlé par la politique.

Les animations générées facilitent l'analyse qualitative des résultats, tout en constituant un support pour illustrer les performances du modèle.

5.4 Résultats générés

Au cours des différents modes d'exécution, le simulateur génère automatiquement un ensemble de fichiers permettant d'analyser les performances des politiques entraînées. Ces fichiers sont organisés selon le mode d'exécution afin de séparer les résultats produits à chaque étape du processus.

Les journaux d'exécution sont enregistrés dans le dossier `logs/`. Celui-ci est organisé en trois sous-dossiers correspondant aux différents modes d'exécution : `train/`, `validation/` et `evaluation/`. Chaque sous-dossier contient les fichiers de résultats associés aux expériences exécutées dans ce mode.

Pour une expérience nommée `<name>`, les fichiers suivants sont générés :

TABLE 15 – Fichiers générés dans le dossier `logs/`.

<i>Fichier</i>	<i>Description</i>
<code><name>_training_metrics.csv</code>	Contient les métriques globales calculées à la fin de chaque épisode (retour cumulé, taux de succès, taux de collision, temps de parcours, vitesses moyennes, etc.). Ce fichier est utilisé pour générer les courbes de performances.
<code><name>_reward_components.csv</code>	Contient les différentes composantes de la fonction de récompense enregistrées au cours de l'entraînement ou de l'évaluation. Il permet d'analyser l'influence de chaque terme de la récompense sur le comportement de la politique.
<code><name>_debug.csv</code>	Disponible uniquement dans le dossier <code>train/</code> . Ce fichier enregistre des informations détaillées à chaque pas de simulation (position, vitesse, distance à l'obstacle, état de l'agent, etc.).

Remarque

Le fichier `<name>_debug.csv` n'est généré que durant l'entraînement. Afin de limiter le volume de données produit, les informations détaillées ne sont pas enregistrées à chaque épisode, mais à une fréquence configurable dans la fonction `run_sac` appelée par `run_train`. Ce fichier est principalement destiné au débogage.

Les fichiers enregistrés dans le dossier `logs/` servent de base à la génération des figures de résultats. Après chaque phase d'entraînement, de validation ou d'évaluation, les métriques sont relues afin de produire les graphiques correspondants, qui sont enregistrés dans le dossier `figures/`. Celui-ci est organisé selon les sous-dossiers `train/`, `validation/`, `evaluation/` et `demo/`.

Pour une expérience nommée `<name>`, les fichiers suivants sont générés :

TABLE 16 – Fichiers générés dans le dossier `figures/`.

<i>Fichier</i>	<i>Description</i>
<code><name>_performances.png</code>	Représente l'évolution des principales métriques de performance au cours des épisodes.
<code><name>_rewards.png</code>	Représente l'évolution des différentes composantes de la fonction de récompense au cours des épisodes.
<code><name>_velocities.png</code>	Représente l'évolution des vitesses linéaires et angulaires moyennes de l'agent au cours des épisodes.
<code><name>.gif</code> et <code><name>.mp4</code>	Animation d'une exécution de la politique sur le scénario considéré.
<code><file_name>_renders.png</code>	Figure comparative présentant l'état initial des différents scénarios, utilisée notamment pour illustrer les environnements d'entraînement et d'évaluation.

Remarque

Les dossiers `train/`, `validation/` et `evaluation/` contiennent les résultats associés à chaque scénario exécuté individuellement. Le dossier `demo/` regroupe quant à lui les visualisations globales, telles que les animations et les figures comparatives des différents scénarios, principalement destinées à l'analyse qualitative et à l'illustration des résultats.

Enfin, les modèles entraînés sont automatiquement sauvegardés dans le dossier `rl/SAC/weights/`. Pour chaque expérience nommée `<name>`, les fichiers suivants sont générés :

TABLE 17 – Fichiers générés dans le dossier `rl/SAC/weights/`.

<i>Fichier</i>	<i>Description</i>
<code><name>_actor.pt</code>	Poids du réseau ActorNetwork.
<code><name>_critic_1.pt</code>	Poids du premier réseau critique CriticNetwork.
<code><name>_critic_2.pt</code>	Poids du second réseau critique CriticNetwork.
<code><name>_target_critic_1.pt</code>	Poids du premier réseau critique cible CriticNetwork.
<code><name>_target_critic_2.pt</code>	Poids du second réseau critique cible CriticNetwork.

Les poids des réseaux de neurones constituent le cœur de la politique apprise. Ils correspondent aux paramètres optimisés au cours de l'entraînement et déterminent directement les actions sélectionnées par l'agent à partir de ses observations. Leur sauvegarde permet de réutiliser une politique sans avoir à la réentraîner, que ce soit pour la validation, l'évaluation, les démonstrations ou la génération d'animations. Elle permet également de reprendre un entraînement à partir d'une politique préalablement apprise.

Glossaire

Cette section rassemble les principaux fichiers et classes du projet afin de faciliter la compréhension de son organisation. Le tableau 18 présente le rôle des différents fichiers et dossiers constituant le simulateur.

TABLE 18 – Glossaire des fichiers du projet.

<i>Dossier ou Fichier</i>	<i>Description</i>
<code>configs/config.py</code>	Définit les différentes classes de configuration utilisées pour représenter les paramètres du simulateur après le chargement des fichiers YAML. Data-classes <code>Experiment</code> , <code>EnvConfig</code> , <code>AgentConfig</code> et <code>RewardConfig</code> .
<code>configs/agents.yaml</code>	Définit les paramètres des agents (rayon, vitesses maximales, taille des observations locales, etc.).
<code>configs/environments.yaml</code>	Définit les différents scénarios de simulation et leurs paramètres.
<code>configs/experiments.yaml</code>	Définit les expériences d’entraînement et d’évaluation en associant les différentes configurations.
<code>configs/rewards.yaml</code>	Définit les paramètres de la fonction de récompense.
<code>figures/demo/</code>	Contient les animations du comportement de la politique sur les différents scénarios et leur représentation à un état initial.
<code>figures/evaluation/</code>	Contient les figures générées durant l’évaluation.
<code>figures/train/</code>	Contient les figures générées durant l’entraînement.
<code>figures/validation/</code>	Contient les figures générées durant la validation.
<code>logs/evaluation/</code>	Contient les métriques enregistrées durant l’évaluation.
<code>logs/train/</code>	Contient les métriques et les informations de débogage enregistrées durant l’entraînement.
<code>logs/validation/</code>	Contient les métriques enregistrées durant la validation.
<code>rl/SAC/weights/</code>	Contient les poids des réseaux de neurones appris durant l’entraînement.

<i>Dossier ou Fichier</i>	<i>Description</i>
<code>rl/SAC/agent.py</code>	Implémente l'agent SAC et les opérations d'apprentissage et d'inférence. Classe <code>SACAgent</code> .
<code>rl/SAC/memory.py</code>	Implémente le replay buffer utilisé pour stocker les transitions d'apprentissage. Classe <code>ReplayBuffer</code> .
<code>rl/SAC/networks.py</code>	Définit les réseaux de neurones utilisés par l'algorithme SAC. Classes <code>FeaturesExtractor</code> , <code>ActorNetwork</code> et <code>CriticNetwork</code> .
<code>rl/SAC/sac.py</code>	Regroupe les fonctions d'entraînement, d'évaluation, de sauvegarde et de chargement des politiques SAC.
<code>simulator/entities/agent.py</code>	Définit la classe représentant les agents mobiles autonomes (AMR). Classe <code>Agent</code> .
<code>simulator/entities/entity.py</code>	Définit la classe de base commune à toutes les entités présentes dans l'environnement. Classe <code>Entity</code> .
<code>simulator/entities/moving_entity.py</code>	Définit les entités mobiles présentes dans l'environnement, notamment les obstacles dynamiques. Classe <code>MovingEntity</code> .
<code>simulator/entities/node.py</code>	Définit les nœuds utilisés par l'algorithme de recherche de chemin A*. Classe <code>Node</code> .
<code>simulator/entities/static_entity.py</code>	Définit les entités statiques présentes dans l'environnement, notamment les obstacles statiques. Classe <code>StaticEntity</code> .
<code>simulator/environment/environment.py</code>	Implémente l'environnement de simulation et la boucle d'interaction avec les agents. Classe <code>Environment</code> .
<code>simulator/environment/gridmap.py</code>	Gère la représentation discrète de l'environnement et les observations locales. Classe <code>GridMap</code> .
<code>simulator/environment/manager.py</code>	Génère et initialise les différents scénarios de simulation. Classe <code>Manager</code> .
<code>simulator/environment/pathfinding.py</code>	Implémente l'algorithme A* utilisé pour calculer les trajectoires des obstacles dynamiques. Classe <code>AStar</code> .
<code>simulator/environment/rewards.py</code>	Implémente la fonction de récompense utilisée durant l'apprentissage.

<i>Dossier ou Fichier</i>	<i>Description</i>
<code>simulator/environment/terminations.py</code>	Détermine les conditions de terminaison et de troncature des épisodes.
<code>simulator/geometry.py</code>	Regroupe les fonctions géométriques utilisées par le simulateur (distances, intersections, angles, etc.).
<code>simulator/spaces.py</code>	Définit les espaces d'observations et d'actions du simulateur.
<code>utils/config_loader.py</code>	Charge les fichiers YAML et instancie automatiquement les classes de configuration correspondantes.
<code>utils/logging.py</code>	Enregistre les métriques dans les fichiers CSV.
<code>utils/plotting.py</code>	Génère les figures, animations et représentations graphiques des résultats.
<code>runner.py</code>	Regroupe les fonctions correspondant aux différents modes d'exécution.
<code>main.py</code>	Point d'entrée du simulateur permettant de sélectionner et de lancer un mode d'exécution.
<code>analysis.ipynb</code>	Notebook d'analyse permettant d'exploiter les journaux d'exécution et de réaliser des analyses complémentaires des résultats.

Au-delà de l'organisation des fichiers, le simulateur repose sur un ensemble de classes interagissant pour gérer la simulation, la génération des scénarios et l'apprentissage par renforcement. Le Tableau 19 présente les classes du projet, leur rôle ainsi que les principales méthodes.

TABLE 19 – Glossaire des principales classes du projet.

<i>Classe</i>	<i>Description</i>	<i>Principales méthodes</i>
<code>Entity</code>	Classe abstraite de base commune à toutes les entités présentes dans l'environnement.	<code>collides_with()</code> , <code>render()</code>
<code>StaticEntity</code>	Représente une entité rectangulaire immobile, notamment les obstacles statiques et les murs.	<code>get_bounds_at()</code> , <code>get_distance()</code> , <code>collides_with()</code> , <code>render()</code>

<i>Classe</i>	<i>Description</i>	<i>Principales méthodes</i>
MovingEntity	Représente une entité circulaire capable de se déplacer, notamment les obstacles dynamiques.	bounds_at(), step(), get_relative_position(), get_relative_distance(), get_distance(), collides_with(), render()
Agent	Représente un robot mobile autonome contrôlé par une politique d'apprentissage par renforcement.	step(), render(), get_vision_field()
Environment	Représente l'environnement de simulation conforme à l'interface Gymnasium.	reset(), step(), render(), done()
Manager	Responsable de la génération et de l'initialisation des entités composant un scénario.	generate(), reset(), generate_static_obstacles(), generate_moving_obstacles(), generate_agents()
GridMap	Construit et met à jour la représentation discrète de l'environnement sous forme de grille d'occupation, utilisée pour les observations locales et le calcul des trajectoires.	grid, current_grid, shape, get_local_grid(), world_to_grid(), grid_to_world()
AStar	Implémente l'algorithme de recherche de chemin A* sur une grille d'occupation bidimensionnelle.	inflate_obstacles(), get_valid_neighbors(), find_path(), reconstruct_path()
SACAgent	Implémente l'algorithme Soft Actor-Critic (SAC).	choose_action(), store_transition(), learn(), update_network_params(), save_checkpoints(), load_checkpoints()
ReplayBuffer	Stocke les transitions d'expérience utilisées par l'algorithme SAC et permet leur échantillonnage aléatoire lors de l'apprentissage.	store_transition(), sample()
FeatureExtractor	Encode les observations de l'agent dans un vecteur de caractéristiques latent utilisé par les réseaux acteur et critique.	forward()

<i>Classe</i>	<i>Description</i>	<i>Principales méthodes</i>
ActorNetwork	Implémente l'actor network de l'algorithme SAC.	forward(), sample_normal(), save_checkpoint(), load_checkpoint()
CriticNetwork	Implémente le critic network de l'algorithme SAC.	forward(), save_checkpoint(), load_checkpoint()
Experiment	Regroupe l'ensemble des configurations nécessaires à l'exécution d'une expérience.	-
EnvConfig	Définit les paramètres décrivant un environnement de simulation.	-
AgentConfig	Définit les paramètres physiques et observationnels des agents.	-
RewardConfig	Définit les paramètres de la fonction de récompense.	-

Les tableaux présentés dans cette section offrent une vue d'ensemble de l'architecture logicielle du simulateur. Ils permettent de retrouver rapidement le rôle des différents fichiers et des principales classes, facilitant ainsi la prise en main, la maintenance et l'évolution du projet.

FAQ

Comment entraîner une nouvelle politique ?

Pour entraîner une nouvelle politique, il suffit de définir les expériences dans `experiments.yaml`, de sélectionner le mode `train` dans le fichier `main.py`, puis d'exécuter le programme avec la commande `python main.py`.

Il est également possible d'entraîner uniquement une partie d'un curriculum afin d'en analyser progressivement les performances. Pour cela, il suffit de modifier les paramètres `experiments` et `previous_exp` de la fonction `run_train()` afin de fournir la sous-liste d'expériences à entraîner ainsi que l'expérience dont les poids doivent être chargés.

Par exemple, le code suivant entraîne uniquement les cinq dernières expériences du curriculum :

```
1 run_train(  
2     experiments=train_experiments[5:],  
3     previous_exp=train_experiments[-6],  
4     trained_agent_id="agent_1",  
5 )
```

Quelle est la différence entre la validation et l'évaluation ?

La validation consiste à évaluer chaque politique sur le scénario sur lequel elle a été entraînée afin de vérifier les performances effectivement acquises. L'évaluation applique au contraire une même politique à plusieurs scénarios différents afin d'étudier sa capacité de généralisation.

Quand utiliser `run_demo` ou `run_animation` ?

La fonction `run_demo` permet d'observer rapidement le comportement d'une politique au cours de son exécution, sans enregistrer de fichier. La fonction `run_animation` enregistre quant à elle une animation (GIF et MP4), ce qui facilite l'analyse des résultats ou leur partage.

À quoi sert le paramètre `trained_agent_id` ?

Ce paramètre indique quel agent a été utilisé lors de l'entraînement de la politique. Il permet ainsi de charger les poids correspondants au bon agent lorsque plusieurs agents sont présents dans un scénario.

Où sont enregistrés les résultats ?

Les poids des réseaux sont enregistrés dans le dossier `rl/SAC/weights/`. Les métriques sont sauvegardées dans le dossier `logs/` et les figures ou animations dans le dossier `figures/`. Chaque mode d'exécution possède son propre sous-dossier afin de séparer les résultats : `train/` pour le mode `train`, `validation/` pour le mode `validation`, `evaluation/` pour le mode `evaluation` et `demo/` pour le mode `animation`.

Comment reprendre un entraînement ?

Lorsqu'un curriculum d'apprentissage est utilisé, les poids obtenus à la fin d'une expérience sont automatiquement réutilisés pour initialiser la suivante. Il est également possible de charger manuellement une politique existante en la renseignant dans le paramètre `previous_exp` de la fonction `run_train()`.

En revanche, il n'est pas possible de reprendre un entraînement au milieu d'une expérience. Chaque expérience doit être exécutée jusqu'à son terme (défini par l'attribut `n_steps`) afin que les poids, les métriques et les figures correspondants soient correctement sauvegardés.

Pourquoi certaines configurations semblent-elles ignorer le nombre d'obstacles ?

Certains modes de génération, comme `warehouse`, reposent sur une disposition prédéfinie des obstacles. Dans ce cas, le nombre et les dimensions des obstacles sont automatiquement déterminés afin de respecter la structure du scénario.

Dans les modes de génération aléatoires, comme `random`, le simulateur tente de placer le nombre d'obstacles demandé tout en respectant les contraintes géométriques, notamment la distance minimale définie par le paramètre `margin`. Si aucune configuration valide ne peut être trouvée après le nombre maximal de tentatives autorisées (`max_attempts`), certains obstacles peuvent être omis. Il est donc possible que le scénario généré contienne moins d'obstacles que le nombre initialement demandé.

Comment ajouter un nouveau scénario ?

Il suffit d'ajouter une nouvelle entrée dans `environments.yaml`, puis de la référencer dans une expérience du fichier `experiments.yaml`. Cette expérience doit également être associée à une configuration d'agent et à une fonction de récompense existantes, ou à de nouvelles configurations si nécessaire. Le scénario pourra ensuite être utilisé dans tous les modes d'exécution.

Dans le cadre d'un curriculum d'apprentissage, il est toutefois recommandé d'utiliser la même fonction de récompense pour l'ensemble des expériences. Dans la version actuelle du simulateur, les agents sont également supposés homogènes; il est donc conseillé de conserver une unique configuration d'agent tout au long du curriculum.

Table des figures

1	Architecture fonctionnelle du simulateur	5
2	Diagramme de classes simplifié du simulateur.	6
3	Workflow d'exécution et organisation des résultats générés par le simulateur.	16

Liste des tableaux

1	Description des principaux dossiers et fichiers du projet.	4
2	Description des fichiers de configuration.	8
3	Description des paramètres de <code>Experiment</code>	9
4	Description des paramètres de <code>EnvironmentConfig</code>	10
5	Description des modes de génération.	11
6	Description des paramètres de <code>AgentConfig</code>	13
7	Description des paramètres de <code>RewardConfig</code>	14
8	Description des modes d'exécution.	15
9	Paramètres de la fonction <code>run_train</code>	17
10	Paramètres de la fonction <code>run_validation</code>	18
11	Paramètres de la fonction <code>run_evaluation</code>	18
12	Paramètres de la fonction <code>run_demo</code>	20
13	Paramètres de la fonction <code>plot_renders</code>	21
14	Paramètres de la fonction <code>run_animation</code>	21
15	Fichiers générés dans le dossier <code>logs/</code>	22
16	Fichiers générés dans le dossier <code>figures/</code>	23
17	Fichiers générés dans le dossier <code>rl/SAC/weights/</code>	23
18	Glossaire des fichiers du projet.	24
19	Glossaire des principales classes du projet.	26